TEST DOCUMENTATION

PROJECT: Traduce Señas

TEAM MEMBERS:
Luis Alejandro Duarte Aldana
Juan Mauricio Suaza Solorzano
Juan Camilo Fierro Esquivel

NATIONAL LEARNING SERVICE – SENA
Software Analysis and Development

2026

Table of Contents

Introduction 3
Test objectives 3
Test strategy 3
Test plan 4
Items to test 4
Items out of scope for this iteration 4
Test environment 4
Acceptance and suspension criteria 4
Roles 5
Results of the automated tests 5
Functional test cases 5
Non-functional tests 7
Sign recognition tests 7
Defects found 8
Conclusions 8
References 9

Introduction

The test documentation brings together the strategy, the test plan, the test cases, and the results obtained for the Traduce Señas software. Its purpose is to demonstrate with evidence that the system meets the requirements of the specification report and to identify the aspects that have not yet been verified.

The structure of this document takes as reference the ISO/IEC/IEEE 29119-3 standard on test documentation (International Organization for Standardization [ISO], 2013) and the ISO/IEC 25010 quality model (ISO, 2011).

Test objectives

- Verify that each microservice meets its business logic and security rules.
- Verify that the services integrate correctly through the gateway and events.
- Validate the critical functional requirements (authentication, translation, history, alphabet, statistics).
- Evaluate the non-functional requirements for accessibility, performance, and security.

Test strategy

The test pyramid is applied: many fast unit tests at the base, fewer integration and contract tests in the middle, and few end-to-end tests at the top.

Table 1
Test levels

Level	Objective	Tool	Target
Unit	Domain logic and use cases in isolation, with fake repositories	node:test (Node.js 20)	≥ 80% of lines; ≥ 90% in domain/
Integration	Adapters with real PostgreSQL, MongoDB, and RabbitMQ	node:test + Docker Compose	Happy path and 2–3 errors per adapter
Contract	HTTP responses comply with the OpenAPI contract	YAML contracts from 07-api	100% of public routes
End-to-end	Complete user flows through the gateway	Docker Compose + HTTP client / manual test	Critical flows
Acceptance	Validation with real users (deaf and hearing people)	Guided pilot test	Use cases CU-01 to CU-06
Accessibility	Contrast, target size, and screen readers	Accessibility Scanner, TalkBack, manual review	WCAG 2.1 AA

Level	Objective	Tool	Target
Performance	Latency and performance of critical services	k6	P95 < 300 ms with 50 requests/s

Test plan

Items to test

- api-gateway and the five microservices (branch feat/microservices-split, commit 5a91e66).
- Shared package @traduce/shared.
- Client application on Android and in a desktop browser.

Items out of scope for this iteration

- Load tests on a production server (does not exist yet).
- Publication in application stores.
- Recognition accuracy of the 20 words in the LSC-54 set (pending the measurement described in the recognition section).

Test environment

Table 2
Environment used

Element	Detail
Equipment	Computer with Windows 11, Node.js 24, and Docker Desktop
Backend	Docker Compose: gateway, 5 services, 5 PostgreSQL 16, MongoDB 7, RabbitMQ 3
Client	Expo web in Chrome; Expo Go on Android
Data	Users and sign test data created by the tests themselves; no real data

Acceptance and suspension criteria

- **Acceptance:** 100% of the automated tests passing and all critical cases passed.
- **Suspension:** execution stops if the build fails or if a critical security test fails.
- **Resumption:** after correcting the defect and re-running the full suite.

Roles

Table 3
Test responsibilities

Role	Responsible	Tasks
Test lead	Juan Mauricio Suaza Solórzano	Planning, backend tests, integration, and reporting
UI tester	Luis Alejandro Duarte Aldana	Screen and accessibility tests
Functional tester	Juan Camilo Fierro Esquivel	Acceptance cases and pilot test

Results of the automated tests

The automated backend suite was run with “npm test”, which builds the shared package and runs the tests of all services with the native Node.js test runner. The result was as follows:

Table 4
Summary of the automated run of October 7, 2026

Indicator	Value
Tests executed	494
Suites (groups)	115
Passed	494
Failed	0
Skipped / cancelled	0 / 0
Total duration	87 seconds

Note. Output of “npm test” on branch feat/microservices-split, commit 5a91e66.

Table 5
Test coverage by component

Component	Test files	Aspects verified
@traduce/shared	5	JWT provider, authMiddleware, requireRole, requirePermission, error handler, configuration, event validators, exponential retry, in-memory bus, and RabbitMQ
api-gateway	1	Prefix-based routing, CORS with allowlist, configuration, public routes
iam-service	22	Registration with email verification, login according to status, password recovery, JWT with roles and permissions, account change and deletion, email templates
recognition-service	11	Translations, history, sign templates (ADMIN only), samples, and consumption of lexicon events with contract
lexicon-service	6	Catalog, alphabet, search, categories, publication, and withdrawal of signs
analytics-service	4	Event logging, reports, and event handlers
profile-service	4	Profile, preferences, notifications, and achievements

Functional test cases

The following cases verify the critical requirements. The “Passed” status indicates that

the case was executed successfully; “Pending” indicates that it has not yet been executed in the corresponding environment.

Table 6
Functional test cases

ID	Requirement	Case	Input data	Expected result	Status
CP-01	FR-001, FR-003	Registration with email verification	New email, valid data, terms accepted	Account INACTIVE; code sent; activated after verification	Passed (automated and E2E)
CP-02	FR-002	Registration without accepting terms	terms_accepted = false	400; account not created	Pending (no specific automated test)
CP-03	FR-005	Valid login	Correct credentials, account ACTIVE	200 with JWT token containing roles	Passed (automated and E2E)
CP-04	FR-005	Login with INACTIVE or BLOCKED account	Unverified or blocked account	Access denied with a clear message	Passed (automated)
CP-05	FR-004, FR-010	Recovery with expired or used code	Code expired / already used	Rejection; attempts are counted	Passed (automated)
CP-06	RNF4	Rate limit on /api/auth	Burst of requests from one IP	429 Too Many Requests	Passed (automated)
CP-07	FR-103	User without ADMIN role imports templates	Token with USER role	403 Forbidden	Passed (automated)
CP-08	FR-401, FR-406	Translation saved to history	Sign recognized, user authenticated	201; appears in history	Passed (E2E with Docker)
CP-09	FR-802	Achievement “First translation”	First translation by a user	Achievement unlocked by the TranslationProduced event	Passed (E2E with Docker)
CP-10	FR-701	Usage event recorded	Translation saved	Event in analytics-service	Passed (E2E with Docker)
CP-11	FR-405	Sign not recognized	Gesture outside the vocabulary	Visual notice; no word is shown	Pending (manual test on device)
CP-12	FR-304, FR-310	Alphabet query	GET /api/lexicon/alphabet	Only LSC letters in ACTIVE status	Passed (automated)
CP-13	FR-406	Private history	User A queries the history of B	Sees only their own translations	Pending (manual test)
CP-14	FR-014	Delete account	Authenticated user	Data deleted in one transaction and usage events anonymized	Passed (automated)
CP-16	FR-602	Sample without consent	Sample without consent field	400; the sample is not saved	Passed (automated)
CP-15	FR-706	Statistics with USER role	Token without permission	403 Forbidden	Passed (automated)

Note. The end-to-end test with Docker (registration → login → translation → achievement and usage event) was executed on October 5, 2026.

Non-functional tests

Table 7
Status of the non-functional tests

Attribute	Test	Criterion	Status
Accessibility	Color contrast of the design system	≥ 4,5:1	Passed: tokens adjusted in the design system
Accessibility	TalkBack navigation in main flows	Complete flows without blocking	Pending
Security	Mandatory JWT secret of 32+ characters	The service does not start without it	Passed (automated)
Security	Databases and broker not exposed to the network	Ports only on 127.0.0.1	Passed (docker-compose review)
Performance	P95 latency of login, translations, and lexicon	< 300 ms with 50 requests/s	Pending (k6)
Performance	Frames per second in capture	≥ 15 FPS on mid-range devices	Pending
Fault tolerance	RabbitMQ outage and recovery	The bus reconnects and retries	Passed (automated)

Sign recognition tests

Recognition accuracy is not measured with unit tests but with an evaluation protocol: samples are taken from people who did not take part in building the templates, the same method used by the application is applied (normalization and Dynamic Time Warping), and for each sign the percentage of correct hits, the percentage of misrecognized words, and the signs with which it is confused are calculated.

For the 20 new words, the public LSC-54 set is used (Mora-Zarate et al., 2025), separating training and test signers. The acceptance target is an accuracy of at least 85% before publishing the templates. This measurement is pending because the data download was stopped due to restrictions on the source server.

Defects found

Table 8
Defect log

ID	Description	Severity	Status
DEF-01	The LSC-54 set downloader could skip entire videos without warning	High	Fixed
DEF-02	Achievement counter not atomic: increments were lost with simultaneous translations	Medium	Fixed: atomic update with ON CONFLICT DO UPDATE
DEF-03	Unescaped username in the HTML of the recovery email	High (security)	Fixed: escapeHtml function
DEF-04	GET /api/profile/me failed if the profile did not yet exist	Medium	In verification
DEF-05	lexicon event contract test failed (305 of 306)	Low	Fixed: the current suite has 494 of 494 passing

Note. The defects come from the code and security audit of phase 4 of the microservices migration.

Conclusions

The backend passes its entire automated suite (494 tests, 0 failures), and the critical end-to-end flows work with all services deployed in Docker. Pending items are the k6 performance tests, the screen reader tests, the measurement of recognition accuracy for the new words, and the pilot test with real users, which are scheduled for the next iteration.

References

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). ISO.

International Organization for Standardization. (2013). *Software and systems engineering — Software testing — Part 3: Test documentation* (ISO/IEC/IEEE 29119-3:2013). ISO.

Mora-Zarate, J. E., Garzón-Castro, C. L., & Castellanos Rivillas, J. A. (2025). LSC-54: A landmark-based dataset for Colombian Sign Language. *Data in Brief*. https://doi.org/10.1016/j.dib.2025.112145

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sakoe, H., & Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing, 26*(1), 43–49. https://doi.org/10.1109/TASSP.1978.1163055

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C. https://www.w3.org/TR/WCAG21/